САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Саид Наваф

K3441

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Выполнить привязку сайта, сделанного в лабораторной работе 1, к внешнему API средствами fetch, axios или xhr. Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию, как в примерах лекций по теме «Имитация работы с API». Вариант остаётся прежним: сайт для поиска работы.

Раздел курса: взаимодействие с внешним API, формат данных JSON, имитация работы с закрытым API (авторизация).

Ход работы

1. Моковое API на JSON-сервере

Для имитации сервера использованы json-server версии 0.17.4 и json-server-auth. В каталоге api/ находятся файл данных db.json, файл прав доступа routes.json и package.json со скриптом запуска (npm start, порт 3000). В db.json описаны четыре ресурса: users (пользователи с ролями соискатель и работодатель), vacancies (вакансии), resumes (резюме) и responses (отклики на вакансии). Пароли пользователей хранятся в виде хэшей bcrypt, а сервер сам создаёт и проверяет JWT-токены.

Права доступа заданы числовыми кодами в routes.json: users имеет код 600 (доступ только владельцу), resumes 640 (владелец читает и пишет, вошедшие читают), vacancies 644 (читают все, пишет только владелец), responses 660 (чтение и запись для вошедших). Благодаря этому, например, чужую вакансию изменить нельзя.

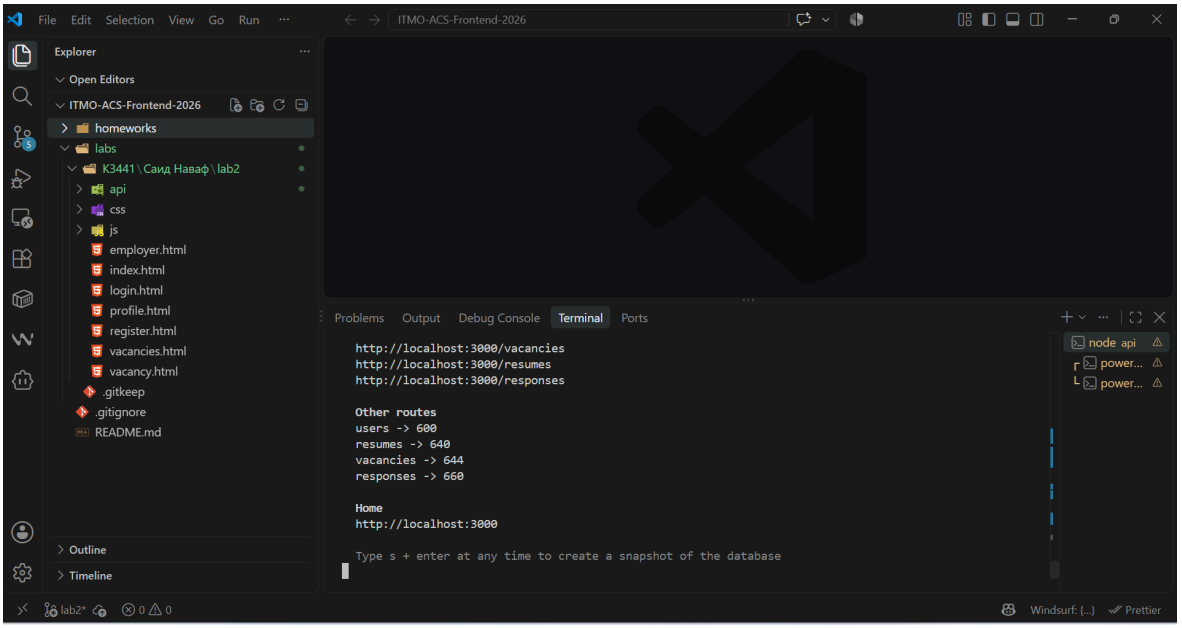


Рисунок 1 – Запуск API (json-server) и список маршрутов с правами доступа

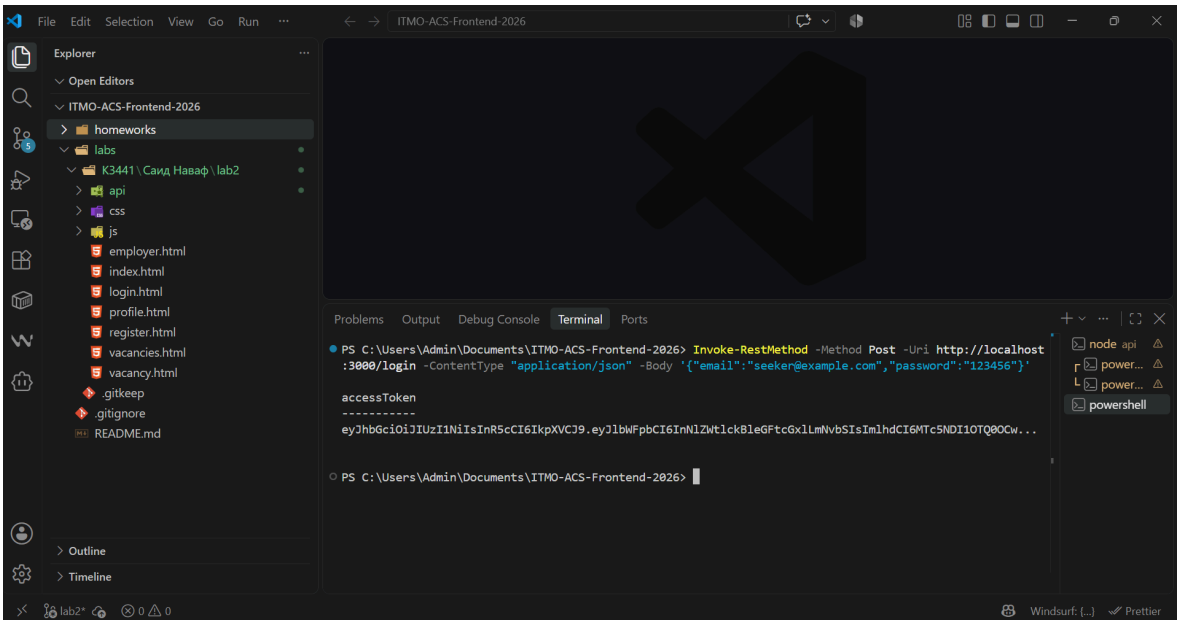


Рисунок 2 – Ответ API на запрос входа: сервер возвращает токен

2. Общий слой работы с API

В js/main.js собран общий код, который подключается на всех страницах. Адрес API вынесен в константу API_URL. Функция apiFetch оборачивает fetch: она добавляет заголовок Authorization с токеном, разбирает JSON-ответ, превращает ошибки сервера в исключения и показывает понятное сообщение, если сервер недоступен. Сообщения

json-server-auth (например, Incorrect password) переводятся на русский язык.

Токен после входа сохраняется в браузере: в localStorage, если выбрано «Запомнить меня», или в sessionStorage. Функция requireAuth закрывает страницы кабинетов: без входа пользователь перенаправляется на страницу входа, а при неподходящей роли в свой кабинет. Шапка сайта после входа показывает имя пользователя и кнопку «Выйти». Справочники отраслей и опыта (INDUSTRIES и EXPERIENCES) используются на нескольких страницах, на сервере хранятся только их короткие коды.

3. Вход и регистрация

Форма входа отправляет запрос POST /login, форма регистрации отправляет POST /register с именем, почтой, паролем и выбранной ролью. Во время запроса кнопка блокируется, а ошибки (неверный пароль, почта уже занята, сервер недоступен) показываются красной плашкой над формой. После успешного входа пользователь попадает в кабинет, соответствующий его роли. Окно восстановления пароля осталось имитацией, так как json-server-auth не поддерживает отправку писем.

4. Поиск вакансий

Карточки вакансий больше не записаны в HTML, а создаются из ответа сервера на запрос GET /vacancies. Фильтры формы превращаются в параметры запроса: industry, experience, salary_gte (минимальная зарплата) и q (поиск по словам). Чтобы не отправлять запрос на каждую букву, используется задержка 300 мс, а ответы на устаревшие запросы игнорируются по номеру запроса. Во время загрузки показывается индикатор, при ошибке сообщение, при пустом результате пояснение. Карточки собираются через createElement и textContent, поэтому данные с сервера не могут внедрить в страницу чужой HTML.

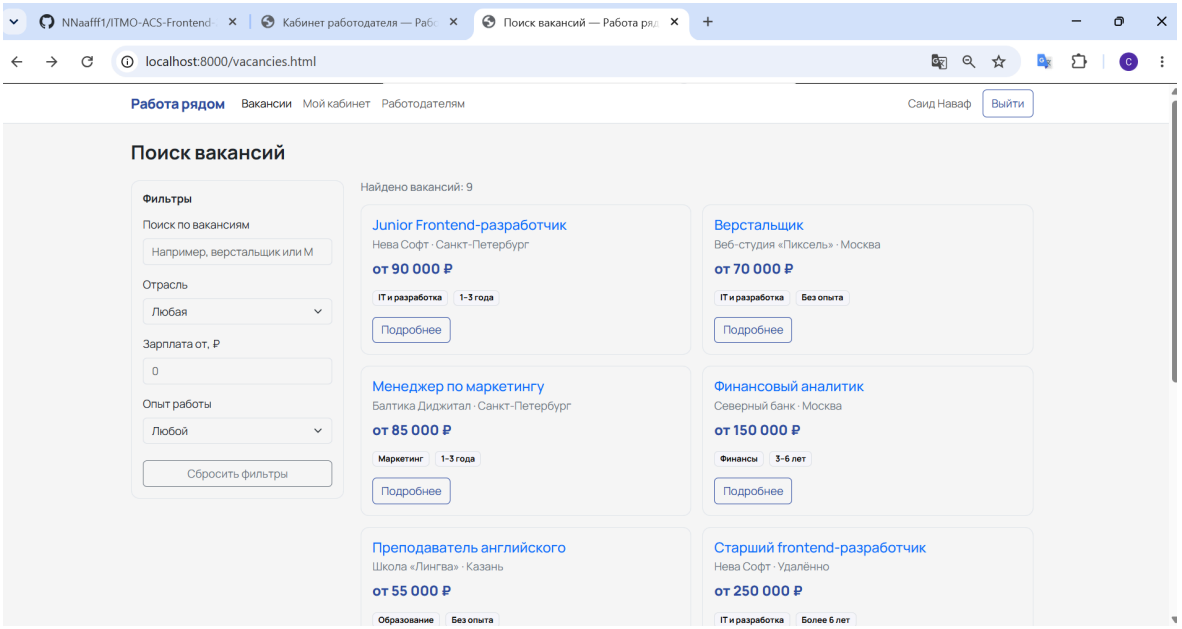


Рисунок 3 – Страница поиска вакансий, данные загружены с API

5. Страница вакансии и отклик

Страница vacancy.html открывается с номером вакансии в адресе (например, vacancy.html?id=1). Скрипт загружает вакансию запросом GET /vacancies/:id и выводит описание, обязанности, требования и условия. Дополнительными запросами загружаются число открытых вакансий компании и две похожие вакансии. Отклик отправляется запросом POST /responses с выбранным резюме и сопроводительным письмом. Без входа кнопка «Откликнуться» ведёт на страницу входа, работодатель откликаться не может, а повторный отклик на ту же вакансию блокируется.

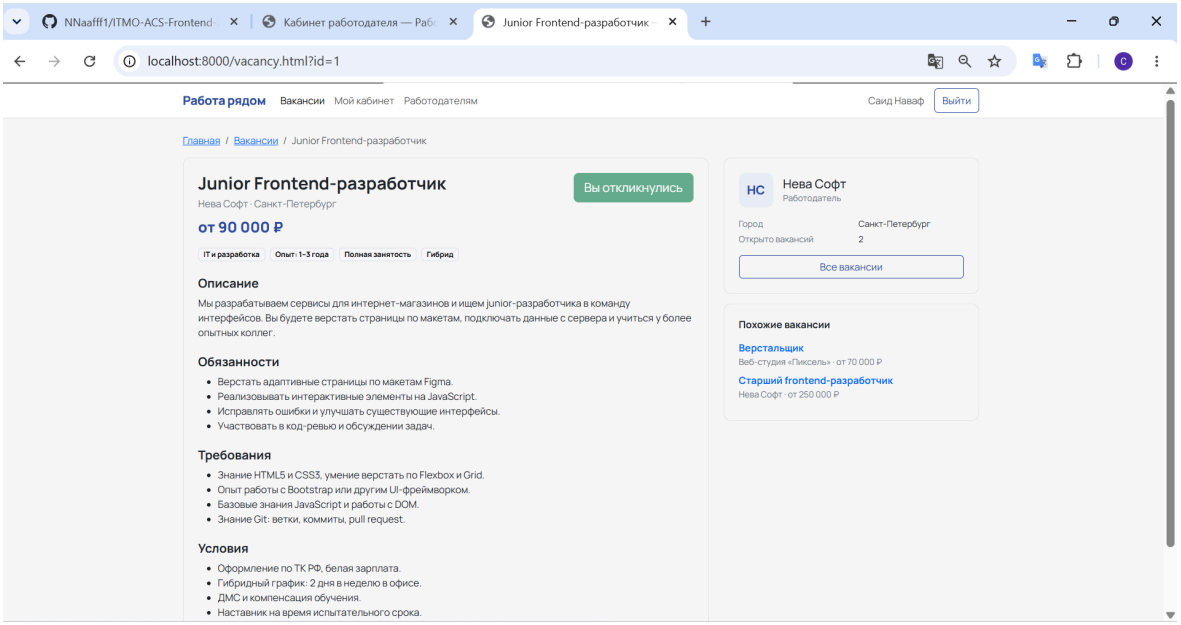


Рисунок 4 – Страница вакансии с данными из API

6. Личный кабинет соискателя

Страница доступна только соискателям. Резюме загружается запросом GET /resumes?userId=, создаётся запросом POST и изменяется запросом PATCH через модальное окно. Если резюме ещё нет, показывается кнопка «Создать резюме». На вкладке «Мои отклики» отклики загружаются запросом GET /responses с параметром _expand=vacancy, поэтому в одной выборке приходят и отклики, и данные вакансий.

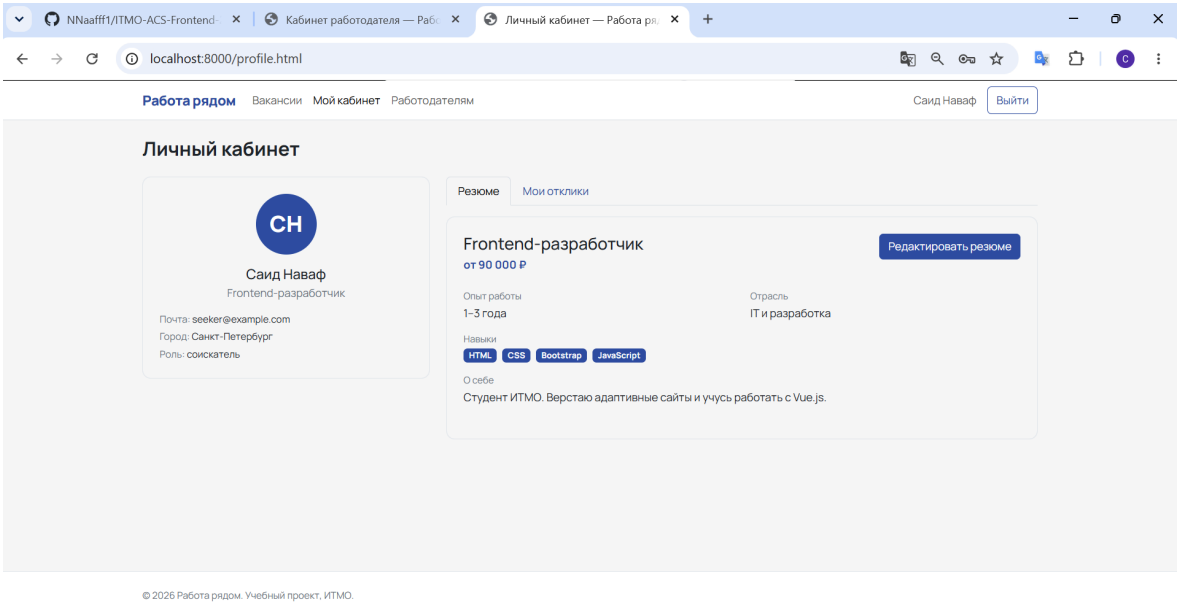


Рисунок 5 – Личный кабинет соискателя с данными из API

7. Личный кабинет работодателя

Страница доступна только работодателям. Вакансии компании загружаются запросом GET /vacancies?userId=&_embed=responses, поэтому число откликов считается из вложенных данных. Новая вакансия создаётся запросом POST, изменение и закрытие вакансии выполняются запросом PATCH, удаление запросом DELETE с параметром _dependent=responses, который удаляет и отклики на вакансию. Карточки со сводкой (активные вакансии, закрытые вакансии, всего откликов) пересчитываются из загруженных данных после каждого действия. Кнопки таблицы обрабатываются одним обработчиком на tbody (делегирование событий).

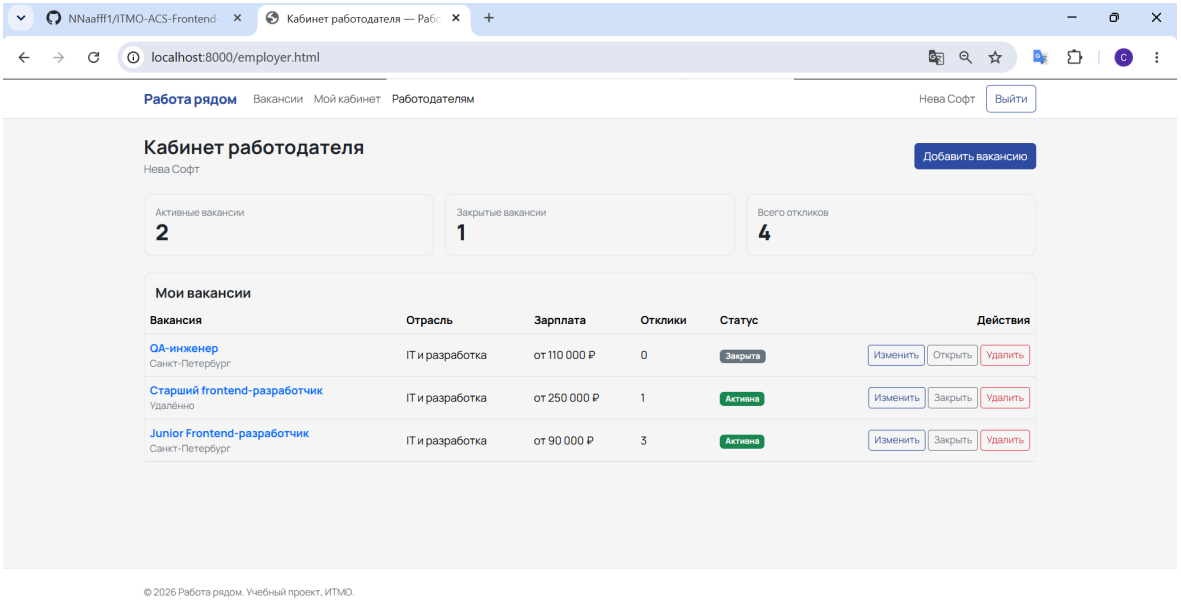


Рисунок 6 – Кабинет работодателя с вакансиями из API

Вывод

В ходе работы сайт для поиска работы из лабораторной работы 1 привязан к API. Реализовано моковое API на json-server с авторизацией json-server-auth: вход и регистрация возвращают JWT-токен, права доступа к данным заданы в routes.json, а страницы кабинетов закрыты для пользователей без нужной роли.

Все данные (вакансии, резюме, отклики) теперь хранятся на сервере и загружаются средствами fetch. Освоены формат JSON, параметры запросов json-server (фильтрация, _embed, _expand, _dependent), хранение токена в браузере, обработка состояний загрузки и ошибок. Ограничения работы: восстановление пароля реализовано как имитация, а данные хранятся в файле db.json без настоящей базы данных.